

# FMA

MDE - Advanced Engineering Topics, ITESO

Gomez Enriquez, Gerardo

[gerardo.gomez@iteso.mx](mailto:gerardo.gomez@iteso.mx)

Guadalajara, Mexico

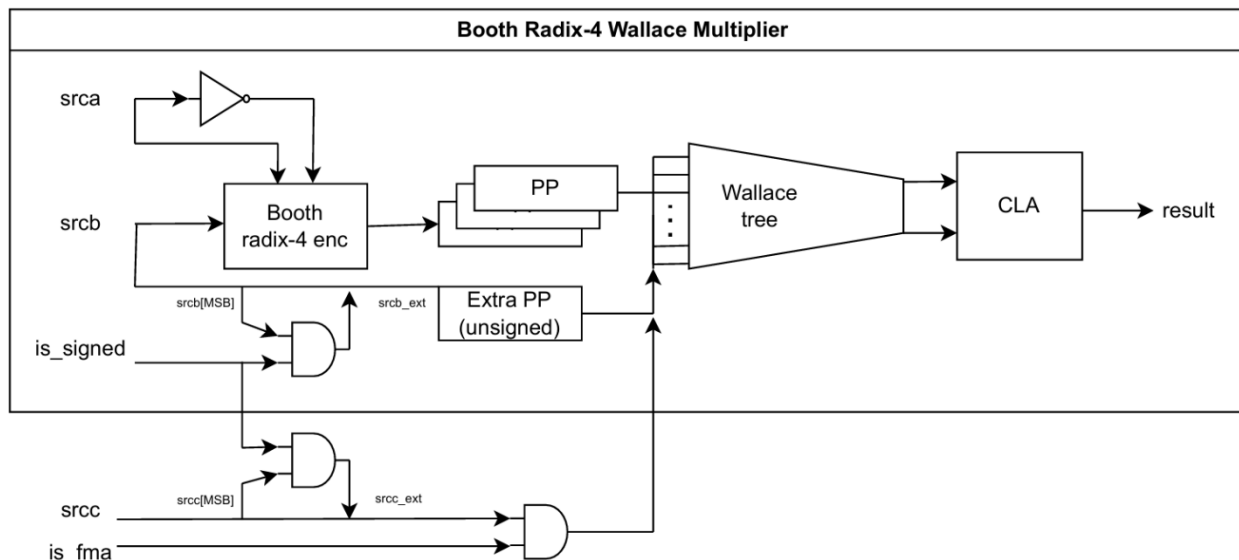
## Architecture and considerations

### Fused Multiply-Add (FMA)

This architecture reuses the Booth Radix-4 Wallace multiplier hardware to compute an add operation on top of the multiplication result with minimal area and typically no timing cost.

Specifically, it takes advantage of the Wallace tree by including the third source (add operation) in the sum of the multiplier partial products. The cost is only an additional “partial product” or operand for the Wallace tree. The CSA levels grow logarithmically, so the impact is null.

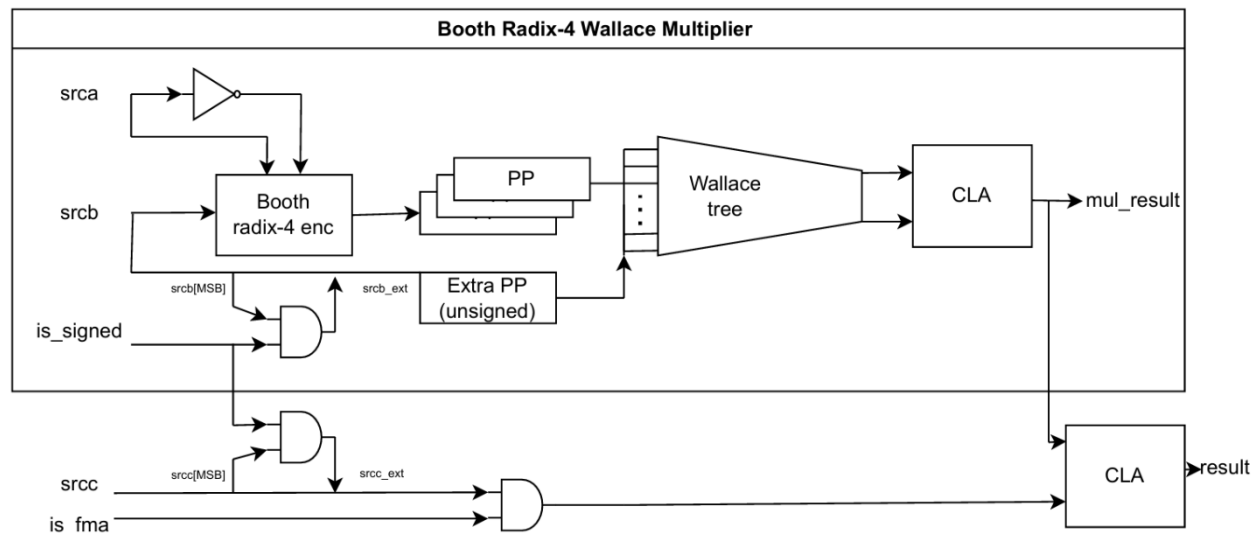
| Width (A×B) | Partial Products (extra PP for unsigned) | Wallace Tree Operands (PP + comp2_corr + FMA srcc) | Wallace CSA Levels | CLA LCU Levels |
|-------------|------------------------------------------|----------------------------------------------------|--------------------|----------------|
| 64×64       | 33                                       | 35                                                 | 8                  | 4              |



## “FMA” forcing multiplication and addition independently

For comparison, this alternative architecture implements first the multiplication operation and then the addition as independent operation, so they are not really “fused”. The cost in area is an additional adder in series with the multiplier, which increases the logic depth drastically making timing worse.

| Width (A×B) | Partial Products (extra PP for unsigned) | Wallace Tree Operands (PP + comp2_corr) | Wallace CSA Levels | CLA LCU Levels (per instance) |
|-------------|------------------------------------------|-----------------------------------------|--------------------|-------------------------------|
| 64×64       | 33                                       | 34                                      | 8                  | 4                             |



## FMA using behavioral model

The final implementation for comparison is using the behavioral model “ $(a * b) + c$ ” for FMA and letting the FPGA synthesis tool decide the architecture / hardware.

## Area / resource usage analysis

### LUT / DSP usage

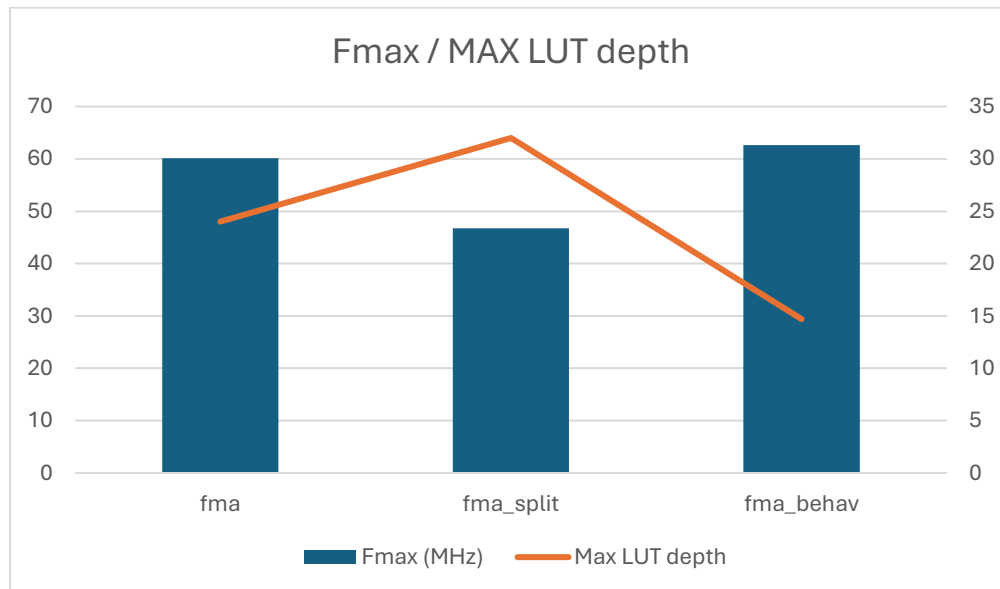
| Resource usage breakdown                        | fma    | fma_split | fma_behav |
|-------------------------------------------------|--------|-----------|-----------|
| <b>Total LUTs</b>                               | 10,056 | 10,262    | 679       |
| <b>arith mode</b>                               | 0      | 0         | 605       |
| — 2 data inputs                                 | —      | —         | 84        |
| — 3 data inputs                                 | —      | —         | 521       |
| <b>normal mode</b>                              | 10,056 | 10,262    | 74        |
| — 0 data inputs                                 | —      | —         | 4         |
| — 1 data input                                  | —      | —         | 2         |
| — 2 data inputs                                 | 630    | 826       | 67        |
| — 3 data inputs                                 | 1,776  | 1,687     | 1         |
| — 4 data inputs                                 | 7,650  | 7,749     | —         |
| <b>DSP (Embedded Multiplier 9-bit elements)</b> | 0      | 0         | 32        |

- *fma* and *fma\_split* (separate multiply then add) both build the entire datapath from soft logic — ~10k 3 and 4 input LUTs in normal mode, zero DSP blocks nor *arith* (carry-chain) cells at all. Folding the addend into the Wallace tree (*fma*) is slightly leaner (10,056 vs 10,262 LUTs) since *fma\_split* pays for a second full-width CLA.
- *fma\_behav* is dominated by arith-mode cells (605 of 679) — Quartus maps the inferred adder and DSP-support logic onto the carry chain, with mostly 3-input arith cells. The multiply itself is absorbed into the 32 DSP elements (16 mac\_mult + 16 mac\_out in the netlist), leaving only a small soft-logic shell.
- The mode split makes the trade-off concrete: the structural designs pay in a large fleet of normal-mode LUTs, while the behavioral design leans on hard multipliers plus a compact arith-mode carry chain.

## Timing / latency analysis

### Fmax & MAX LUT depth

| Implementation   | Fmax      | Max LUT depth |
|------------------|-----------|---------------|
| <b>fma</b>       | 60.15 MHz | 24            |
| <b>fma_split</b> | 46.76 MHz | 32            |
| <b>fma_behav</b> | 62.66 MHz | 14.7          |



- *fma\_behav* is both the fastest (62.66 MHz) and shortest (14.70 depth) — the DSP blocks absorb the multiply, so the critical path is essentially the addend adder plus DSP latency rather than a deep soft-logic tree.
- *fma* (addend folded into the Wallace tree) nearly matches the behavioral Fmax (60.15 MHz) despite using ~15× more LUTs and a much deeper netlist (24.0). Folding srcc as an extra Wallace operand adds it "for free" in terms of depth — it's reduced in parallel with the partial products.
- *fma\_split* is clearly the worst on both metrics (46.76 MHz, depth 32.0). Doing the multiply and the add as two separate serial stages stacks a second full-width CLA after the multiplier, lengthening the critical path by ~8 levels and dropping Fmax ~22% versus *fma*.

## Critical paths

### FMA – Critical path

From: *srcb\_q[1]* → To: *result\_q[127]*

| Node description                | Signal                                                     | Count |
|---------------------------------|------------------------------------------------------------|-------|
| Booth PP-select MUX             | fma Mux*~N combout                                         | 2     |
| Wallace-tree CSA full-adder     | fma wallace_tree gen_lvl[*].gen_csa[*].csa ... fa sum/cout | 6     |
| CLA carry look-ahead unit (LCU) | fma cla gen_lvl[*].gen_lcu[*].lcu4 gg/c                    | 10    |
| CLA final full-adder sum        | fma cla gen_fa[127].fa_gp sum~N                            | 2     |
|                                 | <b>Total</b>                                               | 20    |

### FMA split – Critical path

From: *srcb\_q[3]* → To: *result\_q[127]*

| Node description                       | Signal                                      | Count |
|----------------------------------------|---------------------------------------------|-------|
| Booth PP-select MUX                    | ... multiplier Mux*~N combout               | 2     |
| Multiplier Wallace-tree CSA full-adder | ... multiplier wallace_tree ... fa sum/cout | 7     |
| Multiplier CLA LCU                     | ... multiplier cla ... lcu4 gg/c/pg         | 8     |
| Multiplier CLA final full-adder sum    | ... multiplier cla gen_fa[95].fa_gp sum     | 1     |
| Second adder full-adder propagate      | ... adder gen_fa[95].fa_gp p~2              | 1     |
| Second adder CLA LCU                   | ... adder ... lcu4 gg/c                     | 8     |
| Second adder final full-adder sum      | ... adder gen_fa[127].fa_gp sum             | 1     |
|                                        | <b>Total</b>                                | 28    |

### FMA behavioral – Critical path

From: *srca\_q[63]* → To: *result\_q[127]*

| Node description                                    | Signal                                              | Count           |
|-----------------------------------------------------|-----------------------------------------------------|-----------------|
| Operand sign-extend LUT                             | ... srca_ext[64] combout                            | 1               |
| DSP embedded multiplier (mac_mult + mac_out)        | ... Mult0 auto_generated mac_mult25 / mac_out26     | 2               |
| DSP-output reconstruction adder (arith carry chain) | ... Mult0 auto_generated add53_result[*]~N cout/cin | 60 (arith mode) |
|                                                     | <b>Total</b> (after arith / normal mode weights)    | 8               |

## Other timing parameters

All architectures share similar parameter values:

- **Latency:** 1 cycle (single cycle)
- **Throughput:** 1 operation per cycle
- **Slack:** Significantly negative for both architectures. Frequency was set to 1 GHz for analysis purposes and getting the maximum possible frequency, which was 159.62 MHz for the Booth radix-4 with Wallace tree when widths are 8.

## Conclusion

The comparison between the real fused multiply-add operations and the split / independent multiply and addition operations demonstrate that splitting the addition from the multiplication has no engineering advantage, since both area and logic depth area increased with the additional adder in series.

On the other hand, implementing the FMA describing the hardware in detail vs the behavioral model and letting synthesis decide the FMA architecture / hardware used has a core trade-off when implementing in FPGA: soft logic vs hard silicon.

Data shows that the behavioral model wins on every area / timing metric because it uses dedicated hardware from the multipliers embedded in the DSPs and the adder chain structure in the FPGA architecture. But if those DSPs are limited in the FPGA architecture or if those dedicated multipliers are needed elsewhere, this option evaporates.